

Real-Time Collaborative Editing in Web-Based Educational Documents: A Review of Conflict-Free Synchronization Approaches

Research Paper — Literature-Based Technical Study | 2026

Abstract

Web-based educational documents are increasingly edited by multiple teachers, coordinators, and administrative staff. Conventional save-and-refresh workflows can create lost updates, conflicting versions, and uncertainty about which changes are authoritative. This paper reviews technical approaches for real-time collaborative editing, with particular attention to Operational Transformation (OT), Conflict-Free Replicated Data Types (CRDTs), event-based synchronization, and browser-based rich-text editors. The paper uses a structured literature-based comparison rather than a new empirical experiment. The analysis indicates that CRDT-oriented architectures can provide strong foundations for decentralized conflict handling, while OT remains an important alternative in systems where a centralized transformation service is already established. The paper also argues that transport mechanisms such as WebSockets or Socket.IO should be distinguished conceptually from the collaboration algorithm itself. The resulting design considerations are intended to support future implementation and evaluation of collaborative educational document systems.

1. Introduction

Educational institutions increasingly use browser-based applications to create and maintain documents such as course guides, examination regulations, curricula, and educational catalogues. These documents are rarely authored by one person. A teacher may update learning outcomes while a coordinator changes scheduling information and an examination committee reviews formal requirements. When these actions occur at approximately the same time, a simple last-write-wins persistence model can overwrite legitimate work.

Real-time collaboration attempts to make concurrent editing visible and safe. However, displaying changes in another user's browser is not sufficient. A collaboration system must also decide how concurrent operations are represented, transported, merged, persisted, and recovered after temporary network failures. This paper therefore focuses on the synchronization problem rather than on the visual user interface alone.

2. Research Questions

2.1 Main research question

Which technical approaches can support reliable real-time collaborative editing of structured educational documents in a web application?

2.2 Sub-questions

The analysis considers three sub-questions: (1) how OT and CRDT approaches differ in conflict handling; (2) what role real-time transport layers play in collaboration; and (3) which architectural considerations are important when collaborative editing is integrated with an existing document API and database.

3. Methodology

This paper is a conceptual literature-based study. It synthesizes established publications and technical documentation concerning collaborative editing, distributed data structures, and web communication. No

human participants were recruited and no original benchmark dataset was collected. Consequently, statements about performance or usability are treated as design considerations rather than experimentally verified findings.

The comparison uses four dimensions: conflict resolution, consistency model, implementation complexity, and suitability for rich-text document editing. The purpose is not to produce a numerical ranking, but to identify trade-offs that can guide a future prototype and empirical evaluation.

4. Technical Background

4.1 Operational Transformation

Operational Transformation represents user actions as operations that may be transformed against concurrent operations. The approach became influential in collaborative editors because it can preserve user intent while multiple participants modify a shared document. OT systems generally require careful operation ordering and transformation rules, which can become complex when the document model contains many structural operations.

4.2 Conflict-Free Replicated Data Types

CRDTs represent shared state using data structures designed so that independently generated updates can be merged deterministically. Yjs is an example of a JavaScript CRDT framework that provides shared types such as maps, arrays, and text-like structures. A CRDT can allow clients to continue producing updates while temporarily disconnected, with later synchronization merging compatible state.

4.3 Transport versus Synchronization

WebSocket, Socket.IO, and similar mechanisms provide communication channels; they do not, by themselves, solve concurrent-editing conflicts. A message such as an insert, delete, or document update must still have a defined semantic representation. Separating transport from synchronization logic can therefore make an architecture easier to reason about and test.

4.4 Rich-Text Documents

Rich-text editors introduce additional complexity because the document is not merely a string. Formatting marks, paragraphs, lists, tables, links, and custom components may form a hierarchical structure. Synchronizing complete HTML snapshots can cause unnecessary conflicts because two users may change different parts of the document. Incremental operations or shared structured state can reduce this problem.

5. Comparative Analysis

OT and CRDT approaches both attempt to preserve concurrent user work, but they organize the problem differently. OT commonly relies on transforming operations according to concurrent edits and often fits architectures with a central collaboration service. CRDTs encode merge behavior into the replicated data model, which can make eventual convergence a fundamental property of the representation.

For an educational document application, an important design decision is the granularity of synchronization. Replacing an entire HTML document after every keystroke creates large messages and increases the possibility that one client will overwrite another client's recent change. Synchronizing smaller logical updates can reduce unnecessary data transfer and provides a clearer audit trail of what changed.

A second issue is persistence. A collaboration layer can maintain an in-memory shared document while users are active, but the resulting state must eventually be persisted to the authoritative application database. This creates a boundary between collaboration state and business state. A robust design should define when a shared state is considered committed, how failed persistence is retried, and how a client recovers after reconnecting.

A third issue is authorization. Real-time synchronization must not bypass ordinary application permissions. A user who can view a document but cannot edit a particular section should not gain editing rights merely because a WebSocket connection exists. Authentication, authorization, and collaboration membership therefore need to be explicitly connected.

6. Proposed Architectural Pattern

A practical architecture can separate four responsibilities. First, a browser editor manages local user interaction. Second, a synchronization layer represents and merges collaborative updates. Third, a real-time transport service distributes updates to connected clients. Fourth, the existing API and database remain responsible for durable business data and authorization.

In such an architecture, a client can generate a local update and immediately apply it to its own state. The synchronization layer broadcasts the update to other authorized participants. Each receiving client integrates the update into its local shared state and renders the resulting document. Periodically or at defined transaction boundaries, the shared state can be serialized into the application's persistent document representation.

Testing should include simultaneous insertion, deletion, formatting changes, component movement, reconnects, duplicated messages, delayed messages, and two users modifying adjacent content. These cases are more informative than a test in which only one client edits at a time.

7. Discussion

The literature and technical concepts suggest that there is no universally correct collaboration algorithm. The appropriate choice depends on the document model, server architecture, offline requirements, expected number of participants, and acceptable implementation complexity. For a browser-based application with structured rich text, a CRDT-based prototype can be attractive because it directly addresses concurrent state merging and can operate incrementally.

Nevertheless, CRDT adoption does not automatically solve all collaboration problems. Cursor awareness, permissions, persistence, undo/redo semantics, version history, component identity, and migration between editor representations remain application-level concerns. A technically convergent document can still produce a poor user experience if users cannot understand who changed what or if the application persists an inconsistent business state.

8. Conclusion

Real-time collaboration in educational documents is fundamentally a synchronization and distributed-state problem rather than simply a networking feature. OT and CRDTs provide different mechanisms for handling concurrent operations, while WebSocket-based technologies provide the communication channel through which those operations can be exchanged. For structured rich-text applications, incremental synchronization and explicit separation between collaborative state and durable business state are important architectural principles.

Future work should validate these design considerations through controlled prototype experiments. Useful measurements include convergence time, message volume, recovery after disconnection, conflict frequency, persistence latency, and user-perceived clarity during simultaneous editing.

References

1. Ellis, C. A., Gibbs, S. J., & Rein, G. L. (1991). Groupware: Some issues and experiences. *Communications of the ACM*, 34(1), 39–58.
2. Sun, C., Sun, D., & Chen, Y. (2002). Achieving convergence, causality preservation, and intention preservation in real-time cooperative editing systems. *ACM Transactions on Computer-Human Interaction*, 9(1), 63–108.

3. Shapiro, M., Preguiça, N., Baquero, C., & Zawirski, M. (2011). Conflict-free replicated data types. In *Stabilization, Safety, and Security of Distributed Systems* (pp. 386–400). Springer.
4. Kleppmann, M., & Beresford, A. R. (2017). A conflict-free replicated JSON datatype. *IEEE Transactions on Parallel and Distributed Systems*, 28(10), 2736–2748.
5. Yjs Project. (2026). *Yjs Documentation*. Technical documentation describing shared types, updates, and synchronization concepts.
6. Socket.IO. (2026). *Socket.IO Documentation*. Technical documentation describing event-based real-time communication for web applications.
7. CKSource. (2026). *CKEditor 5 Documentation*. Technical documentation concerning rich-text editing and document models.